

LAPORAN PRAKTIKUM

ALGORITMA DAN PEMROGRAMAN

“PERULANGAN PADA PYTHON”

Disusun Oleh :

Naira Ramadhani Halil

2511533027

Dosen Pengampu:

Wahyudi.DR.. S.T.M.T

Asisten Praktikum:

Muhammad Galid Alvero



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan atas kehadiran Tuhan Yang Maha Esa yang telah memberikan rahmat dan berkahnya, sehingga penulis dapat menyelesaikan laporan praktikum Algoritma dan Pemograman yang telah dilaksanakan sebagai pertemuan ke-5 untuk mata kuliah Praktikum Algoritma dan Pemograman. Praktikum ini berfokus pada pembahasan Perulangan Python.

Melalui praktikum ini, hasil yang diharapkan adalah penulis dapat memahami dan mengimplementasikan secara langsung konsep pemahaman mengenai perulangan

pemrograman menggunakan bahasa Python.

Penulis menyadari bahwa laporan praktikum ini masih memiliki banyak kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan dan pengembangan di masa mendatang.

Padang, 2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Praktikum pemrograman dasar merupakan sarana bagi mahasiswa untuk memahami konsep-konsep fundamental dalam pemrograman, khususnya yang berkaitan dengan struktur kontrol, perulangan, dan pemrosesan input. Salah satu materi yang paling penting dalam pemrograman adalah perulangan (looping), karena hampir seluruh program yang kompleks dibangun di atas kemampuan mengulang suatu proses secara efisien. Pemahaman terhadap perulangan tidak cukup hanya secara teoretis, tetapi perlu dilatih melalui penerapan langsung dalam bentuk kode program.

Pada praktikum ini, mahasiswa diminta untuk menyusun dan menganalisis tujuh kode program yang seluruhnya menggunakan perulangan, mulai dari perulangan sederhana yang menampilkan deret angka, perulangan bersarang (nested loop) untuk membentuk pola dua dimensi, hingga perulangan yang disertai perhitungan akumulatif seperti penjumlahan deret. Setiap kode memiliki karakteristik dan tingkat kesulitan yang berbeda sehingga mahasiswa dapat melihat secara langsung bagaimana perubahan pada batas perulangan, kondisi, dan struktur bersarang mempengaruhi keluaran yang dihasilkan. Dengan demikian, laporan praktikum ini disusun sebagai bentuk dokumentasi sekaligus refleksi atas proses pembelajaran yang telah dilakukan.

1.2 Tujuan

1. Memahami konsep dasar perulangan (for) beserta cara menentukan batas awal, batas akhir, dan langkah perulangan dalam bahasa Python.
2. Menerapkan perulangan bersarang (nested loop) untuk membentuk pola visual dua dimensi, seperti pola segitiga, kotak, dan pola simetris.
3. Menganalisis pengaruh perubahan nilai batas perulangan terhadap jumlah dan susunan keluaran yang dihasilkan program.
4. Memahami penggunaan variabel counter dan akumulator (misalnya perhitungan jumlah deret) di dalam proses perulangan.

5. Melatih penggunaan format keluaran (end, f-string, dan print() kosong) agar output tersusun rapi sesuai pola yang diinginkan.
6. Mengembangkan kemampuan debugging dan penelusuran logika program ketika keluaran tidak sesuai dengan yang diharapkan.

1.3 Manfaat

1. Mahasiswa mampu menulis program perulangan sendiri tanpa harus menyalin kode, sehingga terbentuk pemahaman logika yang lebih kuat.
 2. Mahasiswa terbiasa berpikir sistematis dan terstruktur dalam memecah masalah menjadi langkah-langkah kecil yang dapat diulang.
 3. Mahasiswa memperoleh keterampilan praktis dalam mengontrol alur program, yang menjadi dasar untuk mempelajari struktur data dan algoritma pada mata kuliah lanjutan.
 4. Mahasiswa terlatih membaca, memahami, dan memodifikasi kode orang lain, karena pada praktikum ini setiap program dianalisis satu per satu.
 5. Mahasiswa dapat mengaplikasikan konsep perulangan untuk menyelesaikan kasus nyata, seperti pembuatan pola, perhitungan tabel, dan pengolahan data berulang.
 6. Mahasiswa terbiasa mendokumentasikan hasil kerja dalam bentuk laporan, yang melatih ketelitian dan kemampuan komunikasi teknis.
-

BAB II

PEMBAHASAN

2.1 Dasar Teori

Perulangan (looping) merupakan struktur kontrol yang memungkinkan suatu blok pernyataan dijalankan berulang kali, sehingga penulisan kode menjadi lebih efisien dan tidak redundan. Konsep-konsep dasar yang mendasari ketujuh kode pada praktikum ini antara lain:

1. Perulangan for dan fungsi range() — perulangan for di Python bekerja dengan mengiterasi objek iterable, umumnya hasil range() yang menentukan nilai awal, batas akhir (eksklusif), dan langkah iterasi, seperti `batas_3027 + 1` agar batas masukan pengguna ikut diproses.
2. Perulangan bersarang (nested loop) — perulangan di dalam perulangan, di mana outer loop mengontrol baris dan inner loop mengontrol kolom, sehingga dapat membentuk pola dua dimensi.
3. Parameter end pada print() — mengganti karakter akhir keluaran agar karakter dapat dicetak dalam satu baris, yang merupakan kunci pembentukan pola.

4. Struktur kondisi (if, elif, else) — menentukan karakter apa yang dicetak pada posisi tertentu, terutama untuk pola simetris dan pengecekan bilangan genap.
5. Variabel akumulator — variabel yang nilainya diperbarui bertahap (jumlah = jumlah + nilai) untuk menjumlahkan deret secara iteratif.
6. Operator aritmatika dan modulus (%) — digunakan untuk memanipulasi indeks, menghitung lebar pola $((2 * \text{tinggi}) - 2)$, dan memeriksa kondisi genap/ganjil.
7. String formatting (f-string) — menyisipkan nilai variabel ke dalam teks keluaran secara ringkas, misalnya `f"Perulangan ke-{i_3027}"`.
8. Input pengguna dan konversi tipe data — `input()` menerima masukan berupa string, lalu dikonversi dengan `int()` agar dapat diproses secara aritmatika.

2.2 Kode Program

2.2.1 Program "nested_for1_2511533027"

```
In [ ]: batas_3027 = int(input("Masukkan nilai batas: "))
for line_3027 in range(1, batas_3027 + 1):
    for j_3027 in range(1, (-1 * line_3027 + batas_3027) + 1):
        print(".", end="")
    print(line_3027)
```

Penjelasan : Kode pertama digunakan untuk membentuk pola segitiga menggunakan karakter titik (.).

1. Program meminta pengguna memasukkan nilai batas dan mengubahnya menjadi tipe data integer agar dapat digunakan dalam perulangan.
2. Perulangan luar (line_3027) berjalan dari 1 sampai nilai batas, dan berfungsi mengontrol jumlah baris yang akan dicetak.
3. Perulangan dalam (j_3027) berjalan dari 1 sampai $(-1 * \text{line_3027} + \text{batas_3027}) + 1$, sehingga jumlah titik pada setiap baris semakin berkurang seiring bertambahnya nomor baris.
4. Karakter titik dicetak dengan `print(".", end="")` agar tetap berada pada baris yang sama.
5. Setelah perulangan dalam selesai, `print(line_3027)` mencetak nomor baris di akhir baris tersebut lalu berpindah ke baris berikutnya.
6. Hasilnya adalah pola segitiga terbalik, di mana baris pertama memiliki titik paling banyak dan baris terakhir paling sedikit.

2.2.2 Program "nested_for2_2511533027"

```
In [ ]: batas_3027 = int(input("Masukkan nilai batas: "))
for i_3027 in range(1, batas_3027 + 1):
    for j_3027 in range(1, batas_3027 + 1):
        print("*", end="")
    print()
```

Penjelasan: Kode kedua digunakan untuk membentuk pola kotak penuh menggunakan karakter bintang (*).

1. Program meminta pengguna memasukkan nilai batas dan mengonversinya menjadi integer.
2. Perulangan luar (i_3027) berjalan dari 1 sampai nilai batas dan mengontrol jumlah baris.
3. Perulangan dalam (j_3027) juga berjalan dari 1 sampai nilai batas dan mengontrol jumlah kolom pada setiap baris.
4. Karena kedua perulangan memiliki batas yang sama, setiap baris akan berisi jumlah bintang yang sama banyak.
5. Karakter bintang dicetak dengan print("*", end="") agar berada dalam satu baris.
6. print() tanpa argumen digunakan untuk berpindah ke baris berikutnya setelah satu baris selesai dicetak.
7. Hasilnya adalah pola persegi/kotak penuh berisi bintang dengan ukuran sesuai nilai batas yang dimasukkan.

2.2.3 Program "nested_for3_2511533027"

```
In [ ]: batas_3027 = int(input("Masukkan nilai batas: "))
for i_3027 in range(batas_3027 + 1):
    for j_3027 in range(batas_3027 + 1):
        print(i_3027+j_3027, end=" ")
    print()
```

Penjelasan: Kode ketiga digunakan untuk menampilkan tabel penjumlahan dua indeks secara berulang.

1. Program meminta pengguna memasukkan nilai batas dan mengonversinya menjadi integer.
2. Perulangan luar (i_3027) berjalan dari 0 sampai nilai batas karena menggunakan range(batas_3027 + 1).
3. Perulangan dalam (j_3027) juga berjalan dari 0 sampai nilai batas.
4. Setiap iterasi mencetak hasil penjumlahan i_3027 + j_3027, sehingga terbentuk nilai yang bertambah pada setiap kolom.
5. Pemisah spasi digunakan pada parameter end=" " agar setiap angka tercetak rapi dalam satu baris.
6. print() digunakan untuk berpindah baris setelah satu baris selesai.
7. Hasilnya adalah matriks penjumlahan indeks, di mana nilai pada setiap sel merupakan hasil penjumlahan nomor baris dan nomor kolomnya.

2.2.4 Program "nested_for4_2511533027"

```
In [ ]: tinggi_3027 = int(input("Masukkan tinggi pola (bilangan genap, misal 10): "))

if tinggi_3027 % 2 != 0:
    print("Tinggi harus bilangan genap!")
else:
```

```

a_3027 = tinggi_3027
c_3027 = a_3027
lebar_3027 = (2 * tinggi_3027) - 2

for i_3027 in range(1, tinggi_3027 + 1):
    b_3027 = c_3027 + 1

    for j_3027 in range(1, lebar_3027 + 1):

        #baris atas dan bawah
        if i_3027 == 1 or i_3027 == tinggi_3027:
            if j_3027 == 1 or j_3027 == lebar_3027:
                print("#", end="")
            else:
                print("=", end="")

        # Baris isi
        else:
            if j_3027 == 1 or j_3027 == lebar_3027:
                print("|", end="")
            else:
                if j_3027 == c_3027:
                    print("<", end="")
                elif j_3027 == b_3027:
                    print(">", end="")
                elif j_3027 == (lebar_3027 - c_3027):
                    print("<", end="")
                elif j_3027 == (lebar_3027 - c_3027 + 1):
                    print(">", end="")
                elif j_3027 > b_3027 and j_3027 < (lebar_3027 - c_3027):
                    print(".", end="")
                else:
                    print(" ", end="")

    print()

# Logika asli java
a_3027 -= 2

if a_3027 <= 0:
    c_3027 = (-a_3027) + 2
else:
    c_3027 = a_3027

```

Penjelasan: Kode keempat digunakan untuk membentuk pola simetris kompleks dengan komponen cermin, yang prosesnya meniru logika asli dari bahasa Java.

1. Program meminta pengguna memasukkan tinggi pola dan memvalidasi bahwa nilainya harus bilangan genap menggunakan $\text{tinggi_3027} \% 2 \neq 0$.
2. Jika tinggi bernilai ganjil, program menampilkan pesan kesalahan dan tidak melanjutkan proses.
3. Variabel `a_3027` dan `c_3027` diinisialisasi dengan nilai tinggi, sedangkan `lebar_3027` dihitung dengan rumus $(2 * \text{tinggi_3027}) - 2$.
4. Perulangan luar (`i_3027`) mengontrol baris, dan perulangan dalam (`j_3027`) mengontrol kolom sepanjang lebar pola.

5. Pada baris atas dan bawah, karakter # dicetak di kedua ujung dan = di bagian tengah, sehingga membentuk bingkai.
6. Pada baris isi, karakter | dicetak di kedua ujung, sedangkan karakter < dan > ditempatkan pada posisi tertentu untuk membentuk pola cermin, dan titik (.) mengisi bagian tengahnya.
7. Setelah setiap baris, nilai a_3027 dikurangi 2; jika hasilnya kurang dari atau sama dengan nol, maka c_3027 dihitung dengan $(-a_{3027}) + 2$, dan jika tidak, c_3027 langsung diisi nilai a_3027.
8. Logika pergeseran nilai ini membuat posisi karakter < dan > bergerak secara simetris dari luar ke dalam lalu kembali ke luar, membentuk pola belah ketupat/berlian di dalam bingkai.

2.2.5 Program "perulangan_for1_2511533027"

```
In [ ]: ulang_3027 = int(input("Masukkan jumlah perulangan: "))

for i_3027 in range(ulang_3027):
    print(f"Perulangan ke-{i_3027}")
```

Penjelasan: Kode kelima digunakan untuk menampilkan teks perulangan sejumlah yang dimasukkan pengguna.

1. Program meminta pengguna memasukkan jumlah perulangan dan mengonversinya menjadi integer.
2. Perulangan for berjalan dari 0 sampai ulang_3027 - 1 menggunakan range(ulang_3027).
3. Setiap iterasi mencetak teks "Perulangan ke-{i_3027}" menggunakan f-string agar nilai variabel dapat langsung disisipkan ke dalam teks.
4. Nomor perulangan dimulai dari 0 sesuai perilaku default range().
5. Hasilnya adalah daftar teks perulangan yang tercetak ke bawah sesuai jumlah yang dimasukkan pengguna.

2.2.6 Program "perulangan_for2_2511533027"

```
In [ ]: ulang_3027 = (int(input("Masukkan jumlah perulangan: ")))
print("Perulangan ke-0 sampai ke-", ulang_3027-1)
for i_3027 in range(ulang_3027):
    print(i_3027, end=" ")
print()
print("Perulangan ke-1 sampai ke-", ulang_3027)
for i_3027 in range(1, ulang_3027+1):
    print(i_3027, end=" ")
```

Penjelasan: Kode keenam digunakan untuk menampilkan dua bentuk rentang perulangan, yaitu dari 0 dan dari 1.

1. Program meminta pengguna memasukkan jumlah perulangan dan mengonversinya menjadi integer.

2. Baris pertama menampilkan keterangan rentang perulangan dari 0 sampai ulang_3027 - 1.
3. Perulangan pertama menggunakan range(ulang_3027) sehingga mencetak angka mulai dari 0 hingga satu kurangnya dari nilai batas.
4. Angka dicetak dalam satu baris menggunakan end=" ", lalu diakhiri dengan print().
5. Baris berikutnya menampilkan keterangan rentang perulangan dari 1 sampai ulang_3027.
6. Perulangan kedua menggunakan range(1, ulang_3027 + 1) sehingga mencetak angka mulai dari 1 hingga nilai batas.
7. Hasilnya memperlihatkan perbedaan perilaku range() bergantung pada nilai awal yang diberikan.

2.2.7 Program "perulangan_for3_2511533027"

```
In [ ]: ulang_3027 = int(input("Masukkan jumlah perulangan: "))

jumlah_3027 = 0
for i_3027 in range(1, ulang_3027 + 1):
    print(i_3027, end=" ")
    jumlah_3027 = jumlah_3027 + i_3027

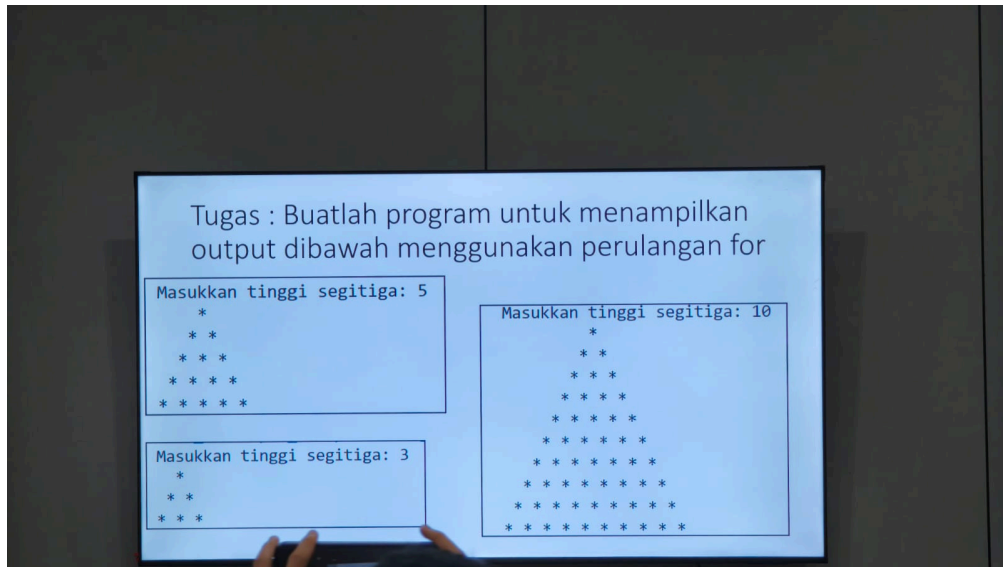
    if i_3027 < ulang_3027:
        print(" + ", end="")
    else:
        print(" = ", jumlah_3027, end="")
print()
print("Jumlah =", jumlah_3027)
```

Penjelasan: Kode ketujuh digunakan untuk menghitung jumlah deret bilangan dari 1 sampai nilai batas yang dimasukkan.

1. Program meminta pengguna memasukkan jumlah perulangan dan mengonversinya menjadi integer.
2. Variabel jumlah_3027 diinisialisasi dengan nilai 0 sebagai akumulator penjumlahan.
3. Perulangan for berjalan dari 1 sampai nilai batas menggunakan range(1, ulang_3027 + 1).
4. Setiap iterasi mencetak angka saat ini dengan end=" ", lalu nilai tersebut ditambahkan ke jumlah_3027 menggunakan jumlah_3027 = jumlah_3027 + i_3027.
5. Kondisi if i_3027 < ulang_3027 digunakan untuk mencetak tanda " + " selama belum mencapai angka terakhir.
6. Jika sudah mencapai angka terakhir, program mencetak tanda " = " beserta nilai totalnya.
7. Setelah perulangan selesai, program menampilkan kembali jumlah akhir deret melalui print("Jumlah =", jumlah_3027).
8. Hasilnya adalah tampilan proses penjumlahan deret secara lengkap beserta totalnya.

Tugas Praktikum

Soal 1:



Gambar 2.1 : Soal Tugas Praktikum

Kode Jawaban :

Kode Program "Tugas5(Segitiga Bintang)_2511533027"

```
In [ ]: tinggi_3027 = int(input("Masukkan tinggi segitiga: "))

for i_3027 in range(1, tinggi_3027 + 1):

    # Membuat spasi di sebelah kiri
    for j_3027 in range(tinggi_3027 - i_3027):
        print(" ", end=" ")

    # Membuat segitiga penuh
    for j_3027 in range(1, (2 * i_3027)):
        print("*", end=" ")

    print()
```

Penjelasan:

1. Program meminta pengguna memasukkan nilai tinggi segitiga dan mengubahnya menjadi tipe data integer agar dapat digunakan dalam perulangan.
2. Perulangan luar (i_3027) berjalan dari 1 sampai nilai tinggi, dan berfungsi mengatur jumlah baris yang akan dicetak.
3. Perulangan pertama (j_3027) digunakan untuk mencetak spasi di bagian kiri agar bentuk segitiga terlihat berada di tengah.
4. Jumlah spasi pada setiap baris semakin berkurang seiring bertambahnya nomor baris, sehingga posisi bintang semakin melebar ke bawah.
5. Perulangan kedua (j_3027) digunakan untuk mencetak karakter *, dengan jumlah bintang sebanyak $2 * i_{3027} - 1$ pada setiap baris.
6. Karakter bintang dicetak menggunakan `print("*", end=" ")` agar tetap berada pada baris yang sama.

7. Setelah kedua perulangan selesai, print() digunakan untuk berpindah ke baris berikutnya.
8. Hasilnya adalah pola segitiga penuh yang semakin melebar dari atas ke bawah.

Output :

```
Masukkan tinggi segitiga: 5
      *
     * * *
    * * * * *
   * * * * * *
  * * * * * * *
 * * * * * * * *
PS D:\2026_prakalpro_A_2511533027>
```

Gambar 2.2 : Output Tugas Praktikum

Soal 2:

B. Studi Kasus: Pola Jam Pasir Kristal Palindromik Berbingkai (Framed Palindromic Crystal Hourglass)

Mahasiswa diminta merancang program konsol interaktif untuk mencetak pola bangun geometri dua dimensi berupa Jam Pasir Kristal Palindromik yang terbungkus rapi di dalam bingkai pembatas berpagar. Berbeda dengan pola dasar, program ini menuntut pembentukan deret angka yang bergerak dinamis menurun dan menaik secara palindromik pada setiap baris serta diselaraskan dengan poros kristal di pusat bangun.

Program menerima masukan nilai integer positif N dari pengguna yang merepresentasikan ukuran skala jam pasir (skala angka tertinggi pada puncak dan alas).

Struktur Pembentukan Pola (3 Fase + 2 Bingkai Horizontal):

1. Bingkai Pembatas Horizontal (Border Atas dan Bawah):

- Karakter pagar sudut (#) di ujung kiri dan kanan.
- Karakter garis ganda (=) di bagian tengah sebanyak $(4 * N + 5)$ karakter.

2. Fase 1: Jam Pasir Atas (Reduksi Angka Menurun: Baris N turun s.d. 1):

- Sisi kiri dibatasi garis tegak (|) dan satu spasi padding.
- Spasi penyeimbang kiri sebanyak $2 * (N - \text{baris})$.
- Deret angka mundur dari baris turun ke 1, dipisahkan oleh spasi (contoh baris 4: "4 3 2 1").
- Poros kristal tengah: karakter "<*>".
- Deret angka maju dari 1 naik ke baris, diawali spasi (contoh baris 4: " 1 2 3 4").
- Spasi penyeimbang kanan sebanyak $2 * (N - \text{baris})$.
- Satu spasi padding dan garis tegak (|) di sisi kanan.

3. Fase 2: Poros Titik Pusat Jam Pasir (Titik Nol / Singularity):

- Karakter garis tegak pembatas (|) di kiri.
- Spasi penyeimbang kiri sebanyak $(2 * N + 1)$.
- Karakter poros kristal tunggal "<*>".
- Spasi penyeimbang kanan sebanyak $(2 * N + 1)$.
- Karakter garis tegak pembatas (|) di kanan.

4. Fase 3: Jam Pasir Bawah (Ekspansi Angka Menaik: Baris 1 naik s.d. N):

- Sisi kiri dibatasi garis tegak (|) dan satu spasi padding.
- Spasi penyeimbang kiri sebanyak $2 * (N - \text{baris})$.
- Deret angka mundur dari baris turun ke 1, dipisahkan oleh spasi.
- Poros kristal tengah: karakter "<*>".
- Deret angka maju dari 1 naik ke baris, diawali spasi.
- Spasi penyeimbang kanan sebanyak $2 * (N - \text{baris})$.
- Satu spasi padding dan garis tegak (|) di sisi kanan.

Ketentuan Teknis Program

Aturan Penamaan Variabel: Setiap variabel yang dideklarasikan wajib diakhiri dengan 4 digit terakhir NIM mahasiswa (contoh: n_1234, baris_1234, spasi_1234, angka_1234). Penulisan variabel harus mematuhi kaidah case-sensitive dan bebas reserved keywords.

Masukan Dinamis: Meminta masukan ukuran N via input() dan dikonversi ke tipe data integer menggunakan int().

Struktur Nested Loop Murni: Dilarang keras menggunakan jalan pintas manipulasi string perkalian seperti print("=" * n), print(" " * spasi), atau perkalian list. Seluruh pembentukan border, deret spasi, dan deret angka WAJIB diiterasi menggunakan perulangan for bersarang dengan parameter end="".

Pindah Baris: Gunakan perintah print() tanpa argumen pada akhir setiap baris pola.

C. Contoh Alur Eksekusi Program (I/O)

```
=== PROGRAM JAM PASIR KRISTAL PALINDROMIK (PEKAN 5) ===
```

Masukkan ukuran skala jam pasir (N): 4

Uji Coba 2 (Ukuran N = 3):

```
=== PROGRAM JAM PASIR KRISTAL PALINDROMIK (PEKAN 5) ===
```

Masukkan ukuran skala jam pasir (N): 3

```
#=====#
```

```
| 3 2 1 <*> 1 2 3 |
```

```
| 2 1 <*> 1 2 |
```

```
| 1 <*> 1 |
```

```
| <*> |
```

```
| 1 <*> 1 |
```

```
| 2 1 <*> 1 2 |
```

```
| 3 2 1 <*> 1 2 3 |
```

```
#=====#
```

D. Catatan Tambahan bagi Mahasiswa

- Gunakan interpreter Python standar laboratorium (Python 3.11.x).
- Dilarang keras melakukan plagiarisme atau menyalin kode antarpraktikan. Setiap kode wajib mencerminkan pengerjaan mandiri dengan akhiran variabel 4 digit NIM masing-masing. Pastikan riwayat commit dan push ke repositori GitHub tercatat sebelum batas waktu pengumpulan berakhir.

Gambar 2.3 : Soal Tugas Praktikum

Kode Jawaban :

Kode Program "Tugas5_2511533027"

```
In [ ]: print("=== PROGRAM JAM PASIR KRISTAL PALINDROMIK (PEKAN 5) ===")

n_3027 = int(input("Masukkan ukuran skala jam pasir (N): "))

if n_3027 <= 0:
    print("N harus bilangan positif!")
else:
    # Border atas
    print("#", end="")
```

```

for i_3027 in range(4 * n_3027 + 5):
    print("=", end="")

print("#")

# Fase 1: Jam Pasir Atas
for baris_3027 in range(n_3027, 0, -1):
    print("|", end="")

    # Padding kiri
    print(" ", end="")
    for spasi_3027 in range(2 * (n_3027 - baris_3027)):
        print(" ", end="")

    # Angka menurun
    for angka_3027 in range(baris_3027, 0, -1):
        print(angka_3027, end=" ")

    # Poros kristal
    print("<*>", end="")

    # Angka menaik
    for angka_3027 in range(1, baris_3027 + 1):
        print(" ", end="")
        print(angka_3027, end="")

    # Padding kanan
    for spasi_3027 in range(2 * (n_3027 - baris_3027)):
        print(" ", end="")

    print(" |")

# Fase 2: Poros titik pusat
print("|", end="")

for spasi_3027 in range(2 * n_3027 + 1):
    print(" ", end="")

print("<*>", end="")

for spasi_3027 in range(2 * n_3027 + 1):
    print(" ", end="")

print("|")

# Fase 3: Jam Pasir Bawah
for baris_3027 in range(1, n_3027 + 1):
    print("|", end="")

    # Padding kiri
    print(" ", end="")
    for spasi_3027 in range(2 * (n_3027 - baris_3027)):
        print(" ", end="")

    # Angka menurun
    for angka_3027 in range(baris_3027, 0, -1):
        print(angka_3027, end=" ")

    # Poros kristal

```

```

print("<*>", end="")

# Angka menaik
for angka_3027 in range(1, baris_3027 + 1):
    print(" ", end="")
    print(angka_3027, end="")

# Padding kanan
for spasi_3027 in range(2 * (n_3027 - baris_3027)):
    print(" ", end="")

print(" |")

# Border bawah
print("#", end="")

for i_3027 in range(4 * n_3027 + 5):
    print("=", end="")

print("#")

```

Penjelasan:

1. Program meminta pengguna memasukkan ukuran skala jam pasir (N) dan mengubahnya menjadi tipe data integer.
2. Program melakukan pengecekan untuk memastikan nilai N lebih besar dari 0 sebelum membuat pola.
3. Perulangan digunakan untuk membuat bingkai atas yang terdiri dari karakter # di bagian ujung dan karakter = di bagian tengah.
4. Perulangan fase pertama berjalan dari nilai N sampai 1 untuk membentuk bagian atas jam pasir.
5. Pada setiap baris, program mencetak spasi di sebelah kiri agar posisi angka sesuai dengan bentuk jam pasir.
6. Perulangan angka pertama mencetak angka secara menurun, kemudian karakter <*> digunakan sebagai poros kristal di tengah.
7. Perulangan angka kedua mencetak angka secara menaik sehingga membentuk pola palindromik pada setiap baris.
8. Program kemudian mencetak spasi di sebelah kanan dan karakter | sebagai batas sisi kanan.
9. Setelah fase pertama selesai, program mencetak poros tengah dengan karakter <*> yang dikelilingi spasi dan dibatasi karakter |.
10. Perulangan fase ketiga berjalan dari 1 sampai N untuk membentuk bagian bawah jam pasir dengan pola angka yang semakin melebar.
11. Bagian bawah menggunakan cara yang sama, yaitu angka menurun, poros <*>, kemudian angka menaik.
12. Setelah seluruh pola selesai, program mencetak bingkai bawah menggunakan karakter # dan =.
13. Hasil akhirnya adalah pola jam pasir kristal palindromik yang memiliki bingkai di bagian atas, bawah, kiri, dan kanan.

Output :

```

=== PROGRAM JAM PASIR KRISTAL PALINDROMIK (PEKAN 5) ===
Masukkan ukuran skala jam pasir (N): 3
#####
| 3 2 1 <*> 1 2 3 |
|   2 1 <*> 1 2   |
|     1 <*> 1     |
|       <*>       |
|     1 <*> 1     |
|   2 1 <*> 1 2   |
| 3 2 1 <*> 1 2 3 |
#####

```

Gambar 2.4 : Output Tugas Praktikum

BAB III

KESIMPULAN

Berdasarkan praktikum yang telah dilaksanakan, dapat disimpulkan bahwa perulangan (looping) merupakan konsep fundamental yang menjadi dasar dalam pembentukan berbagai pola keluaran pada program konsol. Ketujuh kode yang dianalisis memperlihatkan bagaimana perulangan for yang dipadukan dengan fungsi range() mampu menghasilkan keluaran berulang secara efisien, mulai dari pola sederhana hingga pola dua dimensi yang kompleks.

Beberapa poin penting yang dapat disimpulkan adalah:

Perulangan for dengan fungsi range() memungkinkan pengendalian jumlah iterasi secara presisi, termasuk penggunaan langkah negatif (range(N, 0, -1)) untuk membentuk pola menurun dan menaik. Perulangan bersarang (nested loop) terbukti efektif dalam membentuk pola dua dimensi, di mana perulangan luar mengontrol baris dan perulangan dalam mengontrol kolom. Parameter end="" pada fungsi print() menjadi kunci dalam menyusun karakter per baris, sedangkan print() tanpa argumen digunakan untuk berpindah baris. Struktur kondisi if, elif, dan else diperlukan untuk menentukan karakter pada posisi tertentu, terutama pada pola simetris seperti jam pasir. Variabel akumulator dan operasi aritmatika memungkinkan perhitungan deret serta manipulasi indeks, sementara pengukuran lebar isi secara dinamis membuat garis pembatas kanan (|) tetap sejajar untuk berbagai nilai N. Pada studi kasus Jam Pasir Kristal Palindromik, kombinasi seluruh konsep tersebut menghasilkan pola geometri yang simetris, dengan poros <*> berada tepat di pusat bangun dan bingkai ##### membatasi bagian atas dan bawah secara rapi. Dengan demikian, praktikum ini tidak hanya melatih kemampuan menulis kode perulangan, tetapi juga menumbuhkan kemampuan berpikir logis, sistematis, dan teliti dalam menyelesaikan permasalahan pemrograman, khususnya dalam menyusun keluaran yang terstruktur dan terukur.

DAFTAR PUSTAKA

[1] Microsoft, "Python in Visual Studio Code," Visual Studio Code Documentation, 2024.
[Online]. Available: <https://code.visualstudio.com/docs/python/python-tutorial>

[2] R. F. Septian, "E-Book Belajar Pemrograman Python Dasar," Repository UNIKOM, 2013.
[Online]. Available: <https://repository.unikom.ac.id/65984/>